

Problem A. 小王的XCPC 之旅

这是本场比赛的签到题, 直接输入字符串比较相加和 k 对比即可。

时间复杂度 $O(Tn)$ 。

Problem B. 小王的密码破解之旅

看样例, 注意到每一位都等于右边所有位之和等价于这个数字的二进制表示全为 1。所以满足条件的数即为 $2^k - 1$ 。

先预处理完所有的 $2^k - 1$ 的数然后再判断在 $[L, R]$ 中有几个即可。(可以顺序查找, 也可以二分查找)

时间复杂度 $O(T)$ 。

Problem C. 小王的CF 与作业大战

观察到题目中的子序列的要求是不要求连续的, 这里我们借用动态规划的思想。

我们用 4 个变量, $cntc, cntcf, cntz, cntzy$ 来分别记录子序列的数量。

接下来, 我们扫描字符串:

如果当前位置是 "c", 那么 $cntc$ 就增加 1

如果当前位置是 "f", 那么 $cntcf$ 就需要增加前缀 "c" 的数量。

对于 "z" 和 "zy" 一样处理。

我们只需要在扫描的过程中比较 "cf" 和 "zy" 的数量即可, 如果 "zy" 比 "cf" 多, 那么我们就直接输出 "NO", 否则在扫描完成后就输出 "YES" 和两者差的绝对值即可。

注意到数据量最大可能超过 $int32$ 的上限, 所以需要开 $long\ long$ 。

时间复杂度 $O(|s|)$ 。

Problem D. 小王的购物策略

因为每个物品肯定是都要买的, 我们可以看作优惠券可以作用的范围是各个后缀。

越前面的优惠券的作用范围一定覆盖后面的券, 因此不妨从后往前做, 每次选择后缀中的最大值使用, 可以使用优先队列来优化。

时间复杂度为 $O(\sum k \log n)$, 可以通过 Easy Version。

注意到优惠券的优惠价格是固定的, 当我们能优惠 w 元时, 使用它一定不劣。我们不妨把价格进行拆分拆成 $\lfloor \frac{p_i}{w} \rfloor$ 个 w 和一个 $p_i \bmod w$, 把后者放在优先队列里面, 前者的数量用一个变量存储, 前者存在就消耗前者, 后者只需要弹出一次即可, 因此时间复杂度为 $O(n \log n)$, 可以通过两个版本。

Problem E. 小王的游戏时间

注意到题目当中让你求最大战斗力的最小值, 且战斗力最大为 10^9 , 而且战斗力满足单调性 (越大即越容易满足条件)

所以我们二分最大的战斗力, 记为 W , 则我们只计入那些 $p_i \leq W$ 的技能。

对于每组任务的 "每个职业分支最多选一个技能" 这个限制, 我们需要用分组背包, 按编号 id_i 分组, 总时间最大只能选到 t' , 看看能不能达成 $\sum p_i > p'$ 。

具体实现使用滚动数组, 准备一个 ndp 数组和一个 dp 数组, 然后状态转移方程为

$$ndp_j = \max(ndp_j, dp_{j-t_i} + p_i);$$

检查 W 是否可行, 如果可行则进一步缩小 W (即为 $R = mid - 1$), 否则扩大 W (即为 $L = mid + 1$)。如果都不行则输出 -1 。

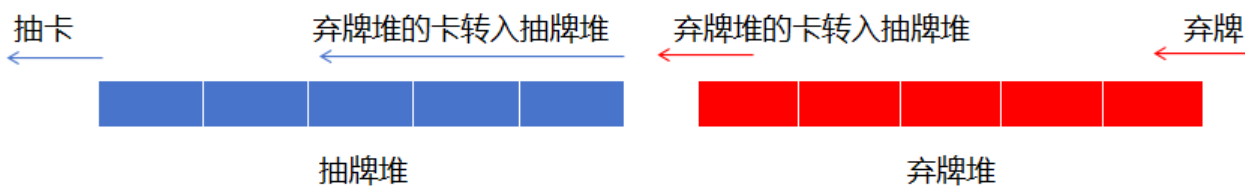
时间复杂度 $O(mt' \log P)$ 。

Problem F. Goodbye 2021

本题为大模拟，按照所给的规则模拟用户操作即可。

游戏中的 n 张卡牌，第一类角色卡， a_i 点生命值和 b_i 点攻击力，抽到时，场上不满 10 张则上场，从左到右按操作顺序放置；结算时，由当前玩家的角色卡从左到右依次对另一名玩家的卡牌，仍遵循从左到右的原则，逐一进行攻击（即维护两个指针，分别表示两组角色卡当前结算的位置），当被攻击的角色卡生命值小于等于 0 时，应立即置于弃牌区，不影响后续的角色卡攻击结算。第二类功能卡，与玩家投骰子的操作对应，相当于“对角色全体造成伤害值”或是“对玩家直接造成伤害”取其一，抽取时立即结算，结算之后立即放入弃牌区。

抽牌堆和弃牌堆的维护，两者都是先进先出的结构，而怎么维护弃牌堆变为抽牌堆的转换，可以将抽牌堆和弃牌堆视作相连的两个队列来便于理解，如下图所示。



那么对于两个牌堆而言，在原有先进先出的基本性质上，均需要维护两种操作，“顶端弹出”和“尾端放入”，普通队列进行维护即可。为了简化代码，也可以用二维数组搭配一个变量 cur 的方式来维护两个牌堆，这样就不需要进行实质上的转换，只是对标记 cur 进行异或 1 的操作，实现 01 转换，从而就能达成两个牌堆之间的置换，将有牌的看做抽牌堆，无牌的看做弃牌堆。由于两个牌堆之间的转换不存在回合轮空，回合内转换后立即抽卡，因此也可以将两个牌堆视作一个牌堆进行处理。

最后，当有一方玩家胜利时，进行记录即可，只需要注意每局游戏的数据的完整读取即可。

时间复杂度 $O(n + k \times 10)$ 。

Problem G. 小王的密码破解之旅2

令原数组为 $a_1, a_2, \dots, a_n$ ，定义前缀异或数组 $P_i = a_1 \oplus a_2 \oplus \dots \oplus a_i$ ($1 \leq i \leq n$)。

则每条线索 (l, r, x) 对应约束 $P_r \oplus P_{l-1} = x \iff P_r = P_{l-1} \oplus x$ 。

则我们只需要找出一组对应满足条件的 P 再还原数组 a 即可。

具体实现我们可以使用带权并查集，因为并查集天然就可以判断是否已经有所冲突，而且很方便合并。

我们对每个结点 i 维护一个他的父亲 f_{a_i} ，以及他们的异或距离 d_i ，保证 $P_i \oplus d_i = P_{f_{a_i}}$ 。这样在 find 操作的时候我们可以同时计算他们的异或值。

对于一次 unite 操作，我们记 $ru = \text{find}(u)$ ， $rv = \text{find}(v)$ ，他们到根的距离分别为 xu 和 xv 即为 $xu = P_u \oplus P_{ru}$ ， $xv = P_v \oplus P_{rv}$ 。

若 $ru \neq rv$ ，则说明可以在并查集合并操作的同时直接设置 $d_{ru} = xu \oplus xv \oplus w$ (即保证 $P_{ru} \oplus d_{ru} = P_{rv}$)。

若 $ru = rv$ ，则说明之前已经出现过相同的祖先情况，我们需要检查是否冲突，即检查是否满足 $xu \oplus xv = w$ ，若不等可直接输出 -1。

在所有合并操作结束之后即可使用 $a_i = P_i \oplus P_{i-1}$ 构造一组可行解。

时间复杂度 $O((n + q)\alpha(n))$ 。

Problem H. 小王的建筑设计

注意到原题可以等价于在一个完全无向图中，任意两条 0 边不能共享一个节点，我们可以从动态规划增加节点/边的角度考虑。

令 dp_i 表示有 i 个节点的图满足条件方案数。

则考虑添加第 i 个节点，我们可以有两种选择：

如果与前 $i-1$ 个节点连接的均是 1 边, 那么此处可以提供的贡献即为原本 $i-1$ 个节点满足的方案数即为 dp_{i-1} 。

如果需要提供 0 边 (当然这个节点只能提供有且只有一条 0 边), 我们可以假设它与其中一个节点 j 与他连了 0 边, 那么当与它连接 0 边之后, 对于节点 j 只能与剩下的其他节点都连 1 边, 则方案数为 dp_{i-2} , 由于 $1 \leq j \leq i-1$, 于是这种情况的总数即为 $(i-1)dp_{i-2}$ 。

两种情况加起来即可求得状态转移方程为 $dp_i = dp_{i-1} + (i-1)dp_{i-2} (i \geq 2)$ 。

如果没有节点或者只有一个节点均没有边, 所以唯一的一组方案都满足条件, 即有 $dp_0 = 1, dp_1 = 1$ 。直接递推即可, 时间复杂度 $O(T+n)$ 。

Problem I. 小王的异或问题

对于原题中给的算式 $X_i = i \oplus (i+1) \oplus \cdots \oplus (i+k-1)$ 。

我们维护前缀异或和 $P_i = 0 \oplus 1 \oplus 2 \oplus \cdots \oplus i$, 则有 $X_i = P_{i+k-1} \oplus P_{i-1}$ 。

于是我们需要求的答案就可以这样转化: $\bigoplus_{i=1}^{n-k+1} (P_{i+k-1} \oplus P_{i-1}) = \left(\bigoplus_{j=k}^n P_j \right) \oplus \left(\bigoplus_{j=0}^{n-k} P_j \right)$ 。

接下来我们只需要考虑如何维护 $\bigoplus_{i=a}^b P_i$ 。我们继续考虑维护前缀和, 令 $Q_i = \bigoplus_{j=0}^i P_j$, 则可得题设算式等价于求 $Q_n \oplus Q_{k-1} \oplus Q_{n-k}$ 。

由于数据量很大, 直接模拟显然不行。我们考虑如何高效计算 P_i, Q_i 。

易得 $P_i = 0 \oplus 1 \oplus \cdots \oplus i = \begin{cases} i, & i \bmod 4 = 0, \\ 1, & i \bmod 4 = 1, \\ i+1, & i \bmod 4 = 2, \\ 0, & i \bmod 4 = 3. \end{cases}$ 那么对于 Q_i , 我们打表注意到它是以 8 作为一个周期

(如 $Q_7, Q_{15}, Q_{22} = 0$), 于是我们开始思考 Q_i 是否和 8 相关。

我们令 $q = \lfloor i/8 \rfloor, r = i \bmod 8$, 则有 $Q_i = \bigoplus_{j=0}^i P_j = \left(\bigoplus_{j=0}^{q-1} \left(\bigoplus_{i=0}^7 P_{8j+i} \right) \right) \oplus \left(\bigoplus_{i=0}^r P_{8q+i} \right)$ 。易得

前面一段是 0, 于是我们可以得到对于 $0 \leq j \leq r$, 有 $P_{8q+j} = \begin{cases} 8q+j, & j \bmod 4 = 0, \\ 1, & j \bmod 4 = 1, \\ 8q+j+1, & j \bmod 4 = 2, \\ 0, & j \bmod 4 = 3. \end{cases}$

得到这个之后, 我们可以得到, 若 $j \bmod 4 \in \{0, 2\}$, 则有 $P_{8q+j} = 8q + P_j = 8q \oplus P_j$ (由于 $0 \leq P_j \leq 7$), 而对于 $j \bmod 4 \in \{1, 3\}$, 则 $P_{8q+j} = P_j$ 。

那么我们就可以推导出 Q_i 的公式

$$Q_i = \begin{cases} 8q \oplus \bigoplus_{j=0}^r P_j, & r \in \{0, 1, 4, 5\}, \\ \bigoplus_{j=0}^r P_j, & r \in \{2, 3, 6, 7\}. \end{cases}$$

对于 $\bigoplus_{j=0}^r P_j$ 直接打表/预处理即可, 时间复杂度 $O(T)$ 。

Problem J. 小王的音符填充游戏

最优的答案一定是选中的数字从大到小排序。

首先贪心的选从大到小的 k 个数字, 可能会出现不是 3 的倍数的情况, 我们在原本的数字的基础上进行调整, 选择一些数去掉, 并从原本没有选中的数进行添加。

一定存在一个方案使得去掉的数字数量不超过 2 个。

因为任意选择 3 个整数一定存在一个非空子集和为 3 的倍数。因此当数量达到 3 时我们一定可以选择一个子集 (也就是撤销前后为 3 的倍数的那个子集) 撤销删除, 撤销后一定比撤销前更优, 同时不改变其模 3 的结果。

又因为选择的元素在模 3 意义下是相同的, 所以我们考虑的范围只需要在选中的部分对模 3 的每一种情

况都选择最小的两个数字, 未选中的部分选择最大的两个数字枚举即可。

问题在于如果长度为 k 不满足条件怎么办, 由于任意选择 3 个整数一定存在一个非空子集和为 3 的倍数。当选择的个数小于等于 $k - 3$ 时, 一定存在一个大小不超过 3 的子集加上后还是 3 的倍数, 所以长度为 $k, k - 1, k - 2$ 的时候一定有答案。

当然如果减到 0 还没有方案说明无解。

时间复杂度 $O(300 \times T + n)$ 。

Problem K. 小王的工作任务分配

如果不考虑修改操作, 考虑两种暴力: 第一种, 暴力累加时间复杂度 $O(\frac{nq}{y})$, 第二种维护 y 个前缀和, 预处理复杂度 $O(yn)$ 。

注意到当 $y > \sqrt{n}$ 时使用前者, $y < \sqrt{n}$ 使用后者, 假设 n, q 同阶则时间复杂度为 $O(n\sqrt{n})$, 可以通过 Easy Version。

考虑修改操作, 很容易想到用树状数组代替第二种暴力的前缀和数组, 需要修改 $O(\sqrt{n})$ 个数组, 因此时间复杂度为 $O(n\sqrt{n} \log n)$, 再调整一下块长可以做到 $O(n\sqrt{n} \log n)$ 的时间复杂度, 难以接受。

注意到查询只需要查一个数组, 而修改需要修改 $O(\sqrt{n})$ 个数组, 可以考虑根号平衡, 如果我们有一个 $O(1)$ 修改 $O(\sqrt{n})$ 查询区间和的数据结构, 就可以将时间复杂度优化到 $O(n\sqrt{n})$, 这个数据结构很容易用分块实现。

最后的问题就在于空间复杂度, 题目要求空间在 32MB, 实际上我们分块维护区间和的时候, 对于散点完全可以不用每一个块状数组都进行存储, 而是直接在原数组中累加, 因此 $\sqrt{n}$ 个数组只需要每个存储 $\sqrt{n}$ 个整块的和, 空间复杂度为 $O(n)$, 可以同时通过两个版本了。